GÖRSEL PROGRAMLAMA PROJESİ

KİTAP SATIŞ TAKİP SİSTEMİ

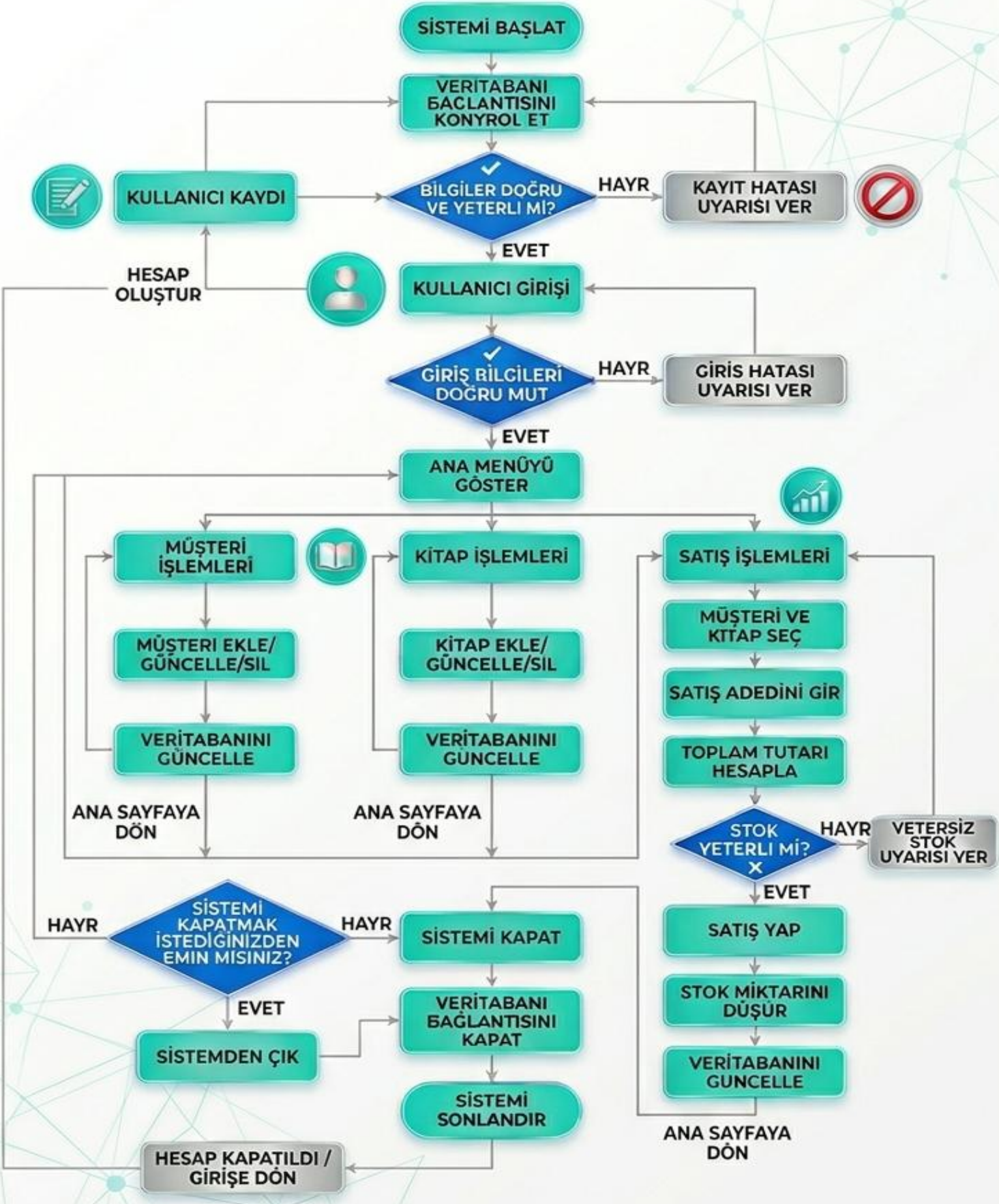


DERSİN SORUMLUSU:
ÖĞR. GÖR. SEZEN GEÇEL ŞENER

HAZIRLAYAN:
BÜŞRA AVCI
NUMARA: 2506022086
BÖLÜMÜ: BİLGİSAYAR PROGRAMCILIĞI (1. SINIF)

2026 - Bahar Dönemi

KİTAP SATIŞ TAKİP SİSTEMİ - AKIŞ ŞEMASI



PROJE TANIMI VE AMACI

Projenin Amacı

Bu proje, kitap satış süreçlerini dijital ortama taşıyarak, geleneksel yöntemlerle yürütülen kayıt işlemlerini otomatize etmek amacıyla geliştirilmiştir. Sistemin temel amacı; kitap stok yönetimi, müşteri takibi ve satış işlemlerini tek bir merkezden, hatasız ve hızlı bir şekilde yürütmektir.

Projenin Kapsamı ve İşleyişi

Geliştirilen "Kitap Satış Takip Sistemi", temel olarak üç ana modülden oluşmaktadır:

- **Kitap İşlemleri:** Sisteme yeni kitapların eklenmesi, mevcut kitap bilgilerinin güncellenmesi ve stoktan düşürülmesi süreçlerini kapsar.
- **Müşteri İşlemleri:** Müşteri kayıtlarının oluşturulması, bilgilerinin düzenlenmesi ve müşteri veri tabanının yönetilmesini sağlar.
- **Satış İşlemleri:** Seçilen müşteri ve kitap bilgileri üzerinden satışın gerçekleştirilmesi, toplam tutarın hesaplanması ve stok miktarının otomatik olarak güncellenmesi sürecini yönetir.

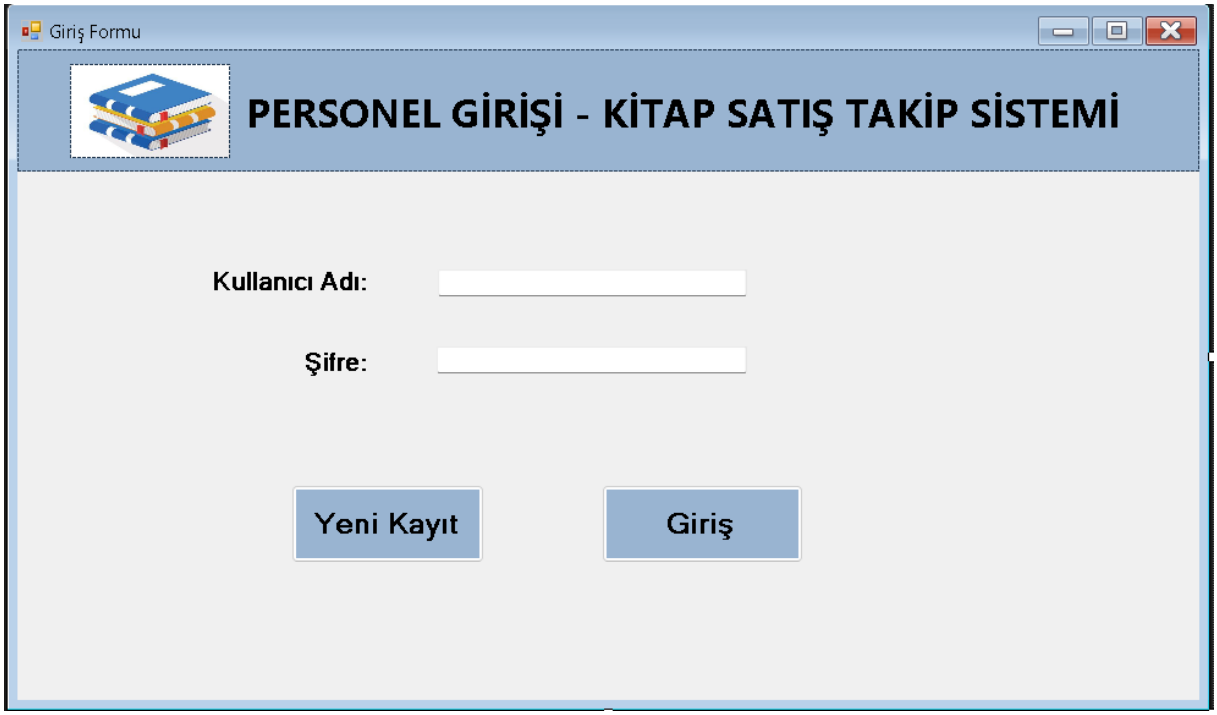
Sistem, kullanıcı girişi ile başlayarak güvenli bir çalışma ortamı sunar. Ana menü üzerinden erişilen tüm modüller, birbiriyle entegre şekilde çalışan bir veri tabanı yapısına bağlıdır. İşlem sonunda kullanıcı, sistemi kapatarak güvenli bir şekilde çıkış yapabilmektedir.

Kullanıcıya Sağladığı Faydalar

- **Zaman Tasarrufu:** Manuel olarak tutulan kayıtların karmaşası ortadan kaldırılarak, verilere saniyeler içinde erişim sağlanır.
- **Hata Payının Azaltılması:** Otomatik stok ve tutar hesaplamaları sayesinde, manuel veri girişinden kaynaklanan hesaplama hataları minimize edilmiştir.
- **Veri Bütünlüğü:** Tüm veriler bir veri tabanı sistemi içerisinde düzenli ve yedeklenebilir formatta saklanarak bilginin kaybolması engellenmiştir.
- **Kullanıcı Dostu Arayüz:** Sade ve anlaşılır tasarım sayesinde, teknik bilgisi kısıtlı kullanıcıların dahi sistemi rahatlıkla kullanabilmesi hedeflenmiştir.

PROJE UYGULAMASI VE TEKNİK DETAYLAR

Giriş Formu (Personel Giriş Ekranı)



Form İşleyişi:

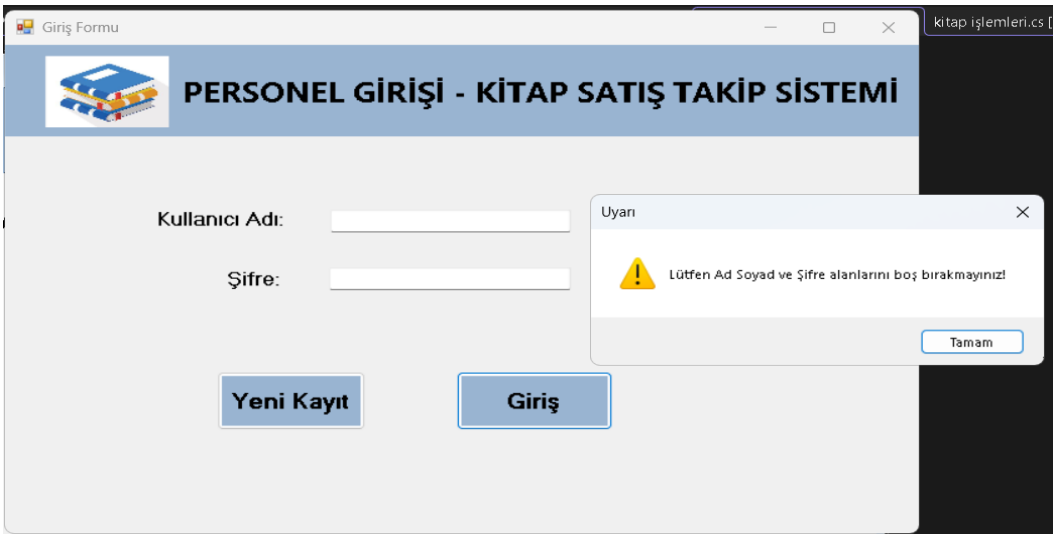
Bu form, sistemin güvenliğini sağlayan ilk kapıdır. Personel, sistem tarafından tanımlanmış "Kullanıcı Adı" ve "Şifre" bilgilerini girerek giriş yapmaya çalışır. Hatalı giriş yapılması durumunda sistem bir uyarı mesajı göstererek erişimi kısıtlar; başarılı girişte ise ana menüye yönlendirme gerçekleştirilir. Ayrıca, sisteme yeni personel eklemek için "Yeni Kayıt" butonu yer almaktadır.

Kullanılan Nesneler:

- **Label:** Kullanıcıyı yönlendirmek için "Kullanıcı Adı" ve "Şifre" metinlerini göstermek amacıyla kullanılmıştır.
- **TextBox:** Personelin bilgilerini (kullanıcı adı ve şifre) güvenli bir şekilde girebilmesi için tasarlanmıştır.
- **Button:** "Giriş" ve "Yeni Kayıt" işlemleri için kullanıcı etkileşimi sağlayan temel araçlardır.
-

Teknik Detaylar (Kod ve Veri tabanı):

Sisteme giriş yapılırken, veri tabanındaki kullanıcı tabloları ile girilen veriler karşılaştırılır. Bu işlem için C# içerisinde **SqlCommand** ve **SqlDataReader** nesneleri kullanılarak veri tabanı sorgulaması yapılır.





İlgili Kaynak Kod ve Çalışma Mantığı

```
1 başvuru
private void btnkayit_Click(object sender, EventArgs e)
{
    Kayıt_Formu kayitEkrani = new Kayıt_Formu();
    this.Hide();
    kayitEkrani.Show();
}
```

Yeni bir kullanıcı hesabı oluşturulması gerektiğinde, Kayıt_Formu nesnesi oluşturularak yeni ekran belleğe yüklenir. Ardından this.Hide() komutu ile mevcut giriş ekranı gizlenir ve kayitEkrani.Show() komutu ile kayıt formu kullanıcıya gösterilir. Bu yöntem, uygulamanın formlar arası geçişlerinde bellek yönetimini optimize etmek ve kullanıcıyı doğru sayfaya yönlendirmek amacıyla tercih edilmiştir.

```
SqlConnection baglanti = new SqlConnection(@"Data Source=BERAT\SQLEXPRESS;Initial Catalog=görsel_proje;Integrated Security=True");

try
{
    baglanti.Open();


    string sorgu = "SELECT * FROM Personeller WHERE (ad + ' ' + soyad) = @adSoyad AND sifre=@sifre";
    SqlCommand komut = new SqlCommand(sorgu, baglanti);

    komut.Parameters.AddWithValue("@adSoyad", giris_ad.Text.Trim());
    komut.Parameters.AddWithValue("@sifre", giris_sifre.Text.Trim());

    SqlDataReader oku = komut.ExecuteReader();
```

 Veritabanı Bağlantısı: SqlConnection sınıfı kullanılarak, görsel_proje veritabanına güvenli bağlantı sağlanmıştır.

 Parametrik Sorgu: Kullanıcıdan alınan veriler doğrudan sorgu içine yazılmamış; SqlCommand ve Parameters.AddWithValue yöntemleri kullanılarak @adSoyad ve @sifre parametrelerine atanmıştır. Bu teknik, **SQL Injection** gibi güvenlik açıklarını önlemek için kullanılan standart bir profesyonel yaklaşımdır.

 Veri Okuma: SqlDataReader nesnesi ile veritabanından dönen kayıtlar sorgulanmış, eşleşen bir veri olması durumunda oku.Read() metodu ile doğrulama işlemi gerçekleştirilmiştir.

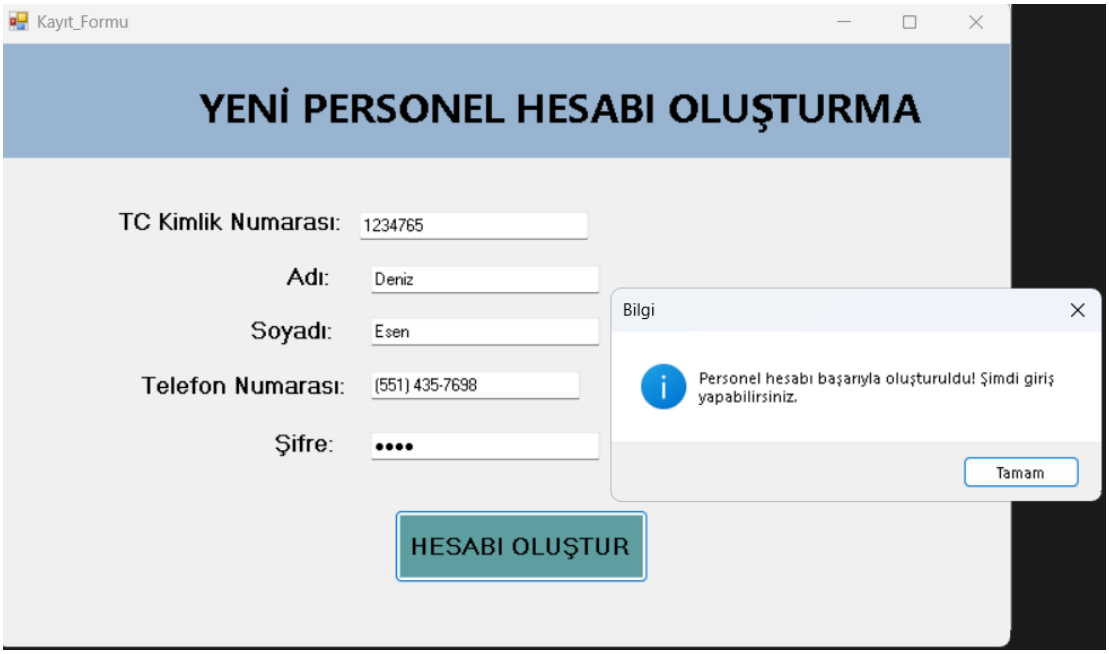
```
    catch (Exception hata)
    {
        MessageBox.Show("Giriş yapılırken sistemsel bir hata oluştu: " + hata.Message, "Sistem Hatası", MessageBoxButtons.OK, MessageBox

        if (baglanti.State == ConnectionState.Open)
        {
            baglanti.Close();
        }
    }
}
```

try-catch bloğu, veri tabanı bağlantısı koptuğunda veya bir işlem hatası yaşandığında programın çökmesini engeller. Hata durumunda bağlantı durumu kontrol edilerek güvenli bir şekilde kapatılır ve kullanıcı hata mesajı ile bilgilendirilir.

Kayıt Formu





Form Açıklaması: Bu form, sisteme yeni personel kaydı eklemek amacıyla tasarlanmıştır. Kullanıcıdan TC Kimlik No, Ad, Soyad, Telefon ve Şifre bilgileri alınır. "Hesabı Oluştur" butonuna basıldığında, girilen veriler doğrulandıktan sonra veri tabanına eklenir ve kullanıcıya işlemin başarıyla tamamlandığına dair bir bilgilendirme mesajı gösterilir.

Kullanılan Nesneler:

- **Label:** Giriş alanlarını tanımlamak (TC, Ad, Soyad vb.) için kullanılmıştır.
- **TextBox:** Personel verilerinin (ad, soyad, şifre) metin olarak alınmasını sağlar.
- **MaskedTextBox:** Telefon numarası gibi standart format gerektiren verilerin doğru formatta girilmesi için tercih edilmiştir.
- **Button:** Veri tabanına kayıt işlemini tetikleyen "Hesabı Oluştur" butonudur.

İlgili Kaynak Kod ve Çalışma Mantığı

```
1 başvuru
private void btn_uye_ol_Click(object sender, EventArgs e)
{

    if (string.IsNullOrWhiteSpace(Tc.Text) ||
string.IsNullOrWhiteSpace(ad.Text) ||
string.IsNullOrWhiteSpace(txtsoyad.Text) ||
string.IsNullOrWhiteSpace(txttel.Text) ||
string.IsNullOrWhiteSpace(sifre.Text))
    {
```

if (string.IsNullOrWhiteSpace(...)) komutu, TC, ad, soyad, tel veya şifre alanlarından herhangi birinin boş olup olmadığını denetler.

Eğer bir eksiklik varsa, MessageBox.Show ile kullanıcıya uyarı mesajı verir.

return; komutu ile işlemin devam etmesini engeller; böylece veritabanına boş veya hatalı kayıt gitmesinin önüne geçer.

```
SqlConnection baglanti = new SqlConnection(@"Data Source=BERAT\SQLEXPRESS;Initial Catalog=görsel_proje;Integrated Security=True");

try
{
    baglanti.Open();

    string sorgu = "INSERT INTO Personeller (tc, ad, soyad, tel, sifre) VALUES (@tc, @ad, @soyad, @tel, @sifre)";
    SqlCommand komut = new SqlCommand(sorgu, baglanti);

    komut.Parameters.AddWithValue("@tc", Tc.Text);
    komut.Parameters.AddWithValue("@ad", ad.Text);
    komut.Parameters.AddWithValue("@soyad", txtsoyad.Text);
    komut.Parameters.AddWithValue("@tel", txttel.Text);
    komut.Parameters.AddWithValue("@sifre", sifre.Text);

    komut.ExecuteNonQuery();
    baglanti.Close();
```

Bağlantı: SqlConnection ile veritabanına bağlanır ve baglanti.Open() ile bu bağlantıyı açar.

Sorgu Hazırlığı: INSERT INTO SQL komutuyla, Personeller tablosundaki sütunlara (tc, ad, soyad, tel, sifre) formdan gelen verilerin ekleneceği belirtilir.

Güvenli Parametreler: komut.Parameters.AddWithValue ile formdaki TextBox verileri, SQL sorgusundaki ilgili parametrelere (örneğin @tc, @ad) güvenli bir şekilde aktarılır.

İşlem: komut.ExecuteNonQuery() komutu, hazırlanan bu sorguyu veritabanında çalıştırarak verileri kalıcı olarak kaydeder.

Kapanış: baglanti.Close() ile veritabanı bağlantısı sonlandırılır.

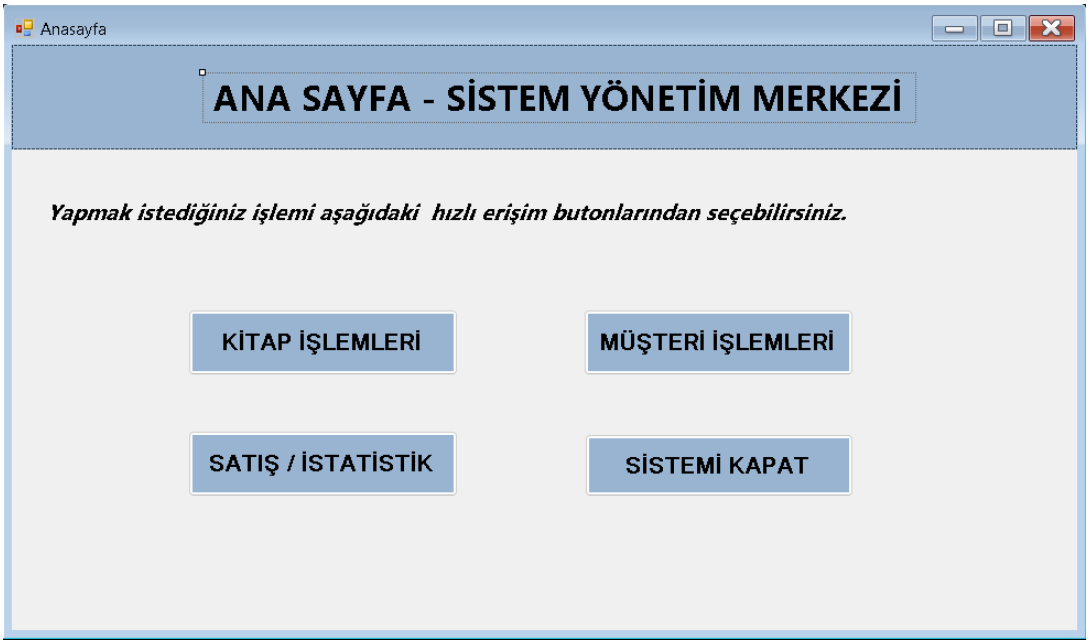
Veritabanı Yapısı ve SQL İşlemleri

```
CREATE TABLE personeller (
    personelID INT IDENTITY PRIMARY KEY,
    tc NVARCHAR(11) NOT NULL,
    ad NVARCHAR(50) NOT NULL,
    soyad NVARCHAR(50) NOT NULL,
    tel NVARCHAR(20) NOT NULL,
    sifre NVARCHAR(50) NOT NULL
);
```

Hesap Oluşturma Formunda: Kullanıcının form ekranına girdiği TC, Ad, Soyad, Telefon ve Şifre bilgileri, bu tablodaki sütunlarla eşleştirilerek INSERT sorgusu ile veritabanına kaydedilir. Tablodaki personelID alanı IDENTITY özelliği sayesinde her yeni personel için otomatik ve benzersiz bir numara atar.

Giriş Formunda: Kullanıcı sisteme giriş yaparken SELECT sorgusu ile bu tablodaki ad, soyad ve sifre sütunları kontrol edilir. Eğer girilen bilgiler tablodaki bir satırla eşleşirse, sistem erişim izni verir.

Ana Sayfa Formu



Form Açıklaması: Anasayfa, uygulamanın ana kontrol merkezidir. Personelin sisteme başarılı giriş yapmasının ardından kullanıcıyı karşılayan bu ekran, tüm operasyonel modüllere (Kitap İşlemleri, Müşteri İşlemleri, Satış/İstatistik) hızlı ve merkezi bir erişim sağlar.

İlgili Kaynak Kod ve Çalışma Mantığı

```
1 başvuru
private void button2_Click(object sender, EventArgs e)
{

    Müşteri_işlemleri musteriEkrani = new Müşteri_işlemleri();
    musteriEkrani.Show();
    this.Hide();
}
```

```
private void btnkitap_Click(object sender, EventArgs e)
{

    kitap_işlemleri kitapEkrani = new kitap_işlemleri();
    kitapEkrani.Show();
    this.Hide();
}

1 başvuru
private void btnsatıs_Click(object sender, EventArgs e)
{

    Satış satisEkrani = new Satış();
    satisEkrani.Show();
    this.Hide();
}

1 başvuru
private void btnkapat_Click(object sender, EventArgs e)
{

    Form1 girisEkrani = new Form1();
    girisEkrani.Show();
    this.Close();
}
```

Her bir buton click (tıklama) olayı, uygulamanın farklı bir modülüne geçişi sağlar. İşlem şu 3 basit adımdan oluşur:

1. **Yeni Nesne Oluşturma:** new anahtar kelimesiyle, gidilmek istenen formun bir kopyası oluşturulur (Örn: new kitap_işlemleri()).
2. **Gösterim:** .Show() metodu ile o form ekrana getirilir.
3. **Gizleme/Kapatma:** this.Hide() ile ana sayfa arka plana alınır (veya this.Close() ile uygulama kapatılır).

- **Kitap, Satış ve Müşteri İşlemleri:** Bu butonlar, kullanıcıyı ilgili modüllere yönlendirir. this.Hide() kullanıldığı için ana sayfayı açık tutar, sadece görünürlüğünü gizler.
- **Sistemi Kapat (btnkapat):** new Form1() ile kullanıcıyı tekrar giriş ekranına gönderir ve this.Close() ile mevcut ana sayfa penceresini tamamen bellekten sonlandırarak güvenli bir çıkış sağlar.

Kitap İşlemleri Formu

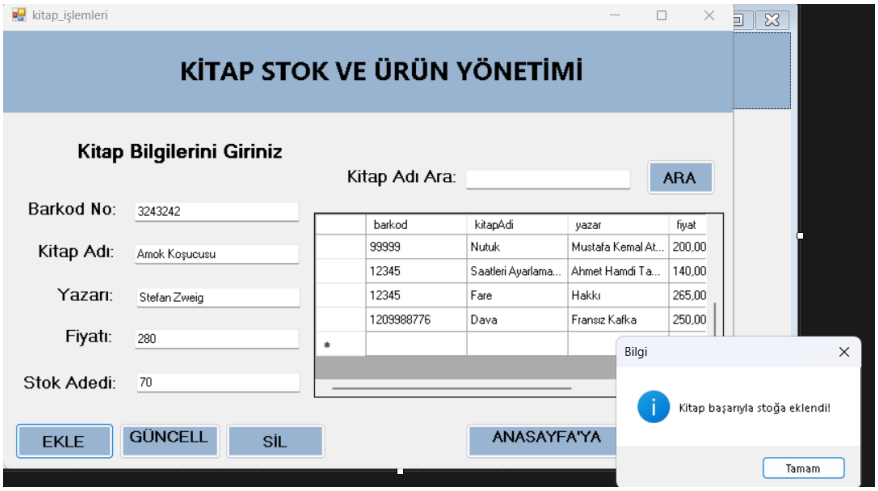


TextBox: Veri girişi ve arama işlemleri için kullanılmıştır.

DataGridView: Kitap kayıtlarının listelenmesi için kullanılmıştır.

Button: İşlemleri (ekle, güncelle, sil, ara) tetiklemek için kullanılmıştır.

Label: Alan tanımlamaları için kullanılmıştır.



Form üzerindeki **TextBox** alanlarına girilen bilgiler, **"EKLE"** butonuna tıklandığında INSERT SQL komutuyla veritabanına gönderilir. Bu işlem, girilen verileri kitaplar tablosuna yeni bir satır olarak kaydeder ve ardından kullanıcıya "Kitap başarıyla stoğa eklendi!" bilgi mesajı iletilir.

"GÜNCELL" butonuna tıklandığında, **TextBox** alanlarında değiştirilen veriler UPDATE SQL komutu ile veritabanındaki ilgili kayıtla eşleştirilerek güncellenir. İşlem sonunda, kullanıcının yapılan değişikliği teyit etmesi için "Kitap bilgileri başarıyla güncellendi!" bilgi mesajı görüntülenir.





"SİL" butonuna tıklandığında, **DataGridView** üzerinde seçili olan kayıt, DELETE SQL komutu aracılığıyla veritabanından kalıcı olarak kaldırılır. İşlem başarıyla tamamlandığında, kullanıcıya işlemin gerçekleştiğini teyit eden "Kitap başarıyla silindi!" bilgi mesajı iletilir.

"Kitap Adı Ara" alanına girilen metin ile **"ARA"** butonuna basıldığında, SELECT sorgusu ve WHERE koşulu kullanılarak veritabanında filtreleme yapılır. Bu işlem, aranan kriterle eşleşen kayıtları **DataGridView** üzerinde listeler.



İlgili Kaynak Kod ve Çalışma Mantığı

```
private void Listele()
{
    SqlConnection baglanti = new SqlConnection(@"Data Source=BERAT\SQLEXPRESS;Initial Catalog=görsel_proje;Integrated Security=True");
    try
    {
        if (baglanti.State == ConnectionState.Closed) baglanti.Open();

        SqlDataAdapter da = new SqlDataAdapter("SELECT barkod, kitapAdi, yazar, fiyat, stok FROM Kitaplar", baglanti);
        DataTable dt = new DataTable();
        da.Fill(dt);
        dataGridView1.DataSource = dt;

        baglanti.Close();
    }
    catch (Exception hata)
    {
        MessageBox.Show("Kitaplar listelenirken hata oluştu: " + hata.Message, "Hata", MessageBoxButtons.OK, MessageBoxIcon.Error);
        if (baglanti.State == ConnectionState.Open) baglanti.Close();
    }
}
```

Bu kod bloğunun amacı, veritabanındaki **Kitaplar** tablosunda bulunan verileri çekerek arayüzdeki **dataGridView1** üzerinde listelemektir.

İşleyiş adımları:

1. **Bağlantı:** SqlConnection ile görsel_proje veritabanına bağlantı açılır.
2. **Veri Çekme:** SqlDataAdapter ile SELECT sorgusu çalıştırılarak veriler alınır.
3. **Aktarım:** Çekilen veriler bir DataTable nesnesine doldurulur ve tablonun veri kaynağı (DataSource) olarak atanır.
4. **Güvenlik:** try-catch yapısı sayesinde olası bağlantı hatalarında programın çökmesi engellenir ve kullanıcıya hata mesajı gösterilir.
5. **Kapanış:** İşlem sonunda bağlantı kapatılarak kaynaklar serbest bırakılır.

```
private void Temizle()
{
    barkod.Clear();
    kita_ad.Clear();
    yazar.Clear();
    fiyat.Clear();
    stok.Clear();
    kitap_ara.Clear();
}
```

Bu Temizle() metodu, form üzerindeki tüm giriş alanlarını sıfırlamak için kullanılır.

Amacı:

- **Kullanıcı Deneyimi:** Her işlemden (ekle, sil, güncelle) sonra, kullanıcının tekrar yeni veri girmesini kolaylaştırmak için eski verileri temizler.
- **Form Yönetimi:** barkod, kita_ad, yazar, fiyat, stok ve kitap_ara kutularının içeriklerini .Clear() komutu ile boşaltarak formu bir sonraki işleme hazır, temiz bir duruma getirir.

```
SqlConnection baglanti = new SqlConnection(@"Data Source=BERAT\SQLEXPRESS;Initial Catalog=görsel_proje;Integrated Security=True");
try
{
    baglanti.Open();

    string sorgu = "INSERT INTO Kitaplar (barkod, kitapAdi, yazar, fiyat, stok) VALUES (@barkod, @ad, @yazar, @fiyat, @stok)";
    SqlCommand komut = new SqlCommand(sorgu, baglanti);


    komut.Parameters.AddWithValue("@barkod", barkod.Text.Trim());
    komut.Parameters.AddWithValue("@ad", kita_ad.Text.Trim());
    komut.Parameters.AddWithValue("@yazar", yazar.Text.Trim());
    komut.Parameters.AddWithValue("@fiyat", Convert.ToDecimal(fiyat.Text));
    komut.Parameters.AddWithValue("@stok", Convert.ToInt32(stok.Text));

    komut.ExecuteNonQuery();
    baglanti.Close();
```

Bu kod bloğu, kullanıcı tarafından girilen kitap bilgilerini veritabanına **güvenli bir şekilde** kaydeder.

İşleyiş:

1. **Bağlantı:** SqlConnection ile veritabanına erişim açılır.
2. **Parametreli Sorgu:** INSERT komutu ile veriler @ işaretli parametrelere atanır. Bu yöntem, veritabanını dışarıdan gelebilecek zararlı SQL komutlarına karşı (SQL Injection) korur.
3. **Dönüşüm:** Convert fonksiyonları ile metin kutularındaki veriler, veritabanındaki sayısal alanlara (Decimal, Int32) uygun formatta dönüştürülür.
4. **Uygulama:** ExecuteNonQuery() komutuyla veriler tabloya yazılır ve bağlantı sonlandırılır.

```
SqlConnection baglanti = new SqlConnection(@"Data Source=BERAT\SQLEXPRESS;Initial Catalog=görsel_proje;Integrated Security=True");
try
{
    baglanti.Open();
    string sorgu = "DELETE FROM Kitaplar WHERE barkod=@barkod";
    SqlCommand komut = new SqlCommand(sorgu, baglanti);

    komut.Parameters.AddWithValue("@barkod", barkod.Text.Trim());

    komut.ExecuteNonQuery();
    baglanti.Close();

    MessageBox.Show("Kitap başarıyla silindi!", "Bilgi", MessageBoxButtons.OK, MessageBoxIcon.Information);

    Temizle();
    Listele();
}
catch (Exception hata)
{
    MessageBox.Show("Kitap silinirken hata oluştu: " + hata.Message, "Hata", MessageBoxButtons.OK, MessageBoxIcon.Error);
    if (baglanti.State == ConnectionState.Open) baglanti.C
}
                                                (yerel değişken) Exception hata
```

Bu kod bloğu, kullanıcı tarafından seçilen veya barkodu girilen bir kitabın veri tabanından **kalıcı olarak silinmesini** sağlar.

İşleyiş:

1. **Bağlantı:** SqlConnection ile veri tabanına bağlantı açılır.
2. **Silme Sorgusu:** DELETE SQL komutu, belirtilen barkod numarasına sahip kaydı hedef alır.
3. **Güvenlik:** @barkod parametresi kullanılarak veri tabanı saldırılarına karşı önlem alınır.
4. **İşlem:** ExecuteNonQuery() komutu ile silme işlemi veri tabanında gerçekleştirilir ve bağlantı kapatılır.
5. **Güncelleme:** İşlem sonrası Temizle() ile giriş kutuları boşaltılır ve Listele() ile güncel veri tablosu ekrana getirilir.

```
SqlConnection baglanti = new SqlConnection(@"Data Source=BERAT\SQLEXPRESS;Initial Catalog=görsel_proje;Integrated Security=True");
try
{
    baglanti.Open();
    string sorgu = "UPDATE Kitaplar SET kitapAdi=@ad, yazar=@yazar, fiyat=@fiyat, stok=@stok WHERE barkod=@barkod";
    SqlCommand komut = new SqlCommand(sorgu, baglanti);

    komut.Parameters.AddWithValue("@barkod", barkod.Text.Trim());
    komut.Parameters.AddWithValue("@ad", kita_ad.Text.Trim());
    komut.Parameters.AddWithValue("@yazar", yazar.Text.Trim());
    komut.Parameters.AddWithValue("@fiyat", Convert.ToDecimal(fiyat.Text));
    komut.Parameters.AddWithValue("@stok", Convert.ToInt32(stok.Text));

    komut.ExecuteNonQuery();
    baglanti.Close();

    MessageBox.Show("Kitap bilgileri başarıyla güncellendi!", "Bilgi", MessageBoxButtons.OK, MessageBoxIcon.Information);

    Temizle();
    Listele();
}
catch (Exception hata)
{
    MessageBox.Show("Güncelleme yapılırken hata oluştu: " + hata.Message, "Hata", MessageBoxButtons.OK, MessageBoxIcon.Error);
    if (baglanti.State == ConnectionState.Open) baglanti.Close();
}
```

Bu kod bloğu, veritabanındaki mevcut bir kitap kaydının bilgilerini **güncellemek** için kullanılır.

İşleyiş:

1. **Bağlantı:** SqlConnection ile veritabanı bağlantısı açılır.
2. **Sorgu:** UPDATE SQL komutu, girilen barkod değerine sahip kaydı bulur ve diğer alanları (kitap adı, yazar, fiyat, stok) yeni verilerle günceller.
3. **Güvenlik:** @ parametreleri kullanılarak veritabanı güvenliği sağlanır.
4. **Uygulama:** ExecuteNonQuery() komutuyla değişiklik veritabanına işlenir ve bağlantı kapatılır.
5. **Geri Bildirim:** İşlem sonrası kullanıcıya onay mesajı gösterilir, Temizle() ile kutular boşaltılır ve Listele() ile tablodaki güncel bilgiler ekrana yansıtılır.

Veritabanı Yapısı ve SQL İşlemleri

```
CREATE TABLE kitaplar (
    kitapID INT IDENTITY PRIMARY KEY,
    barkod NVARCHAR(11) NOT NULL,
    kitapAdi NVARCHAR(70) NOT NULL,
    yazar NVARCHAR(80) NOT NULL,
    fiyat DECIMAL(18,2) NOT NULL,
    stok INT NOT NULL
);
```

```
INSERT INTO kitaplar
(barkod, kitapAdi, yazar, fiyat, stok)
VALUES
('11111', 'Beyaz Geceler', 'Dostoyevski', 120.00, 50),
('22222', 'Küçük Prens', 'Antoine de Saint-Exupéry', 85.00, 30),
('33333', 'Simyacı', 'Paulo Coelho', 150.00, 40),
('44444', 'Suç ve Ceza', 'Dostoyevski', 165.00, 25),
('55555', 'Sefiller', 'Victor Hugo', 190.00, 15),
('66666', 'Kürk Mantolu Madonna', 'Sabahattin Ali', 95.00, 60),
('77777', '1984', 'George Orwell', 110.00, 45),
('88888', 'Hayvan Çiftliği', 'George Orwell', 75.00, 70),
('99999', 'Nutuk', 'Mustafa Kemal Atatürk', 200.00, 100),
('12345', 'Saatleri Ayarlama Enstitüsü', 'Ahmet Hamdi Tanpınar', 140.00, 20);
```

Bu SQL kodları, kitap satış takip sisteminin **veritabanı altyapısını** oluşturur. Yazdığınız C# kodundaki tüm işlemler bu yapı üzerinde gerçekleşir.

1. Tablo Oluşturma (CREATE TABLE)

kitaplar tablosunu oluşturur.

- **Bağlantı:** C# tarafındaki INSERT, UPDATE ve DELETE sorgularının hedeflediği veri yapısını tanımlar.
- **Özellik:** kitapID alanı otomatik artan (IDENTITY) bir anahtar olarak tanımlanmıştır; böylece her kitap sistemde benzersiz bir kimliğe sahip olur.

2. Veri Ekleme (INSERT INTO)

Tabloyu hazır örnek verilerle doldurur.

- **Bağlantı:** Uygulama ilk açıldığında Listele() metodu çalıştırıldığında, bu verilerin dataGridView1 üzerinde görünmesini sağlar.
- **Özellik:** Barkod, kitap adı, yazar, fiyat ve stok bilgileri eşleştirilerek tabloya eklenir; böylece kullanıcı boş bir ekranla karşılaşmaz.

Müşteri İşlemleri Formu

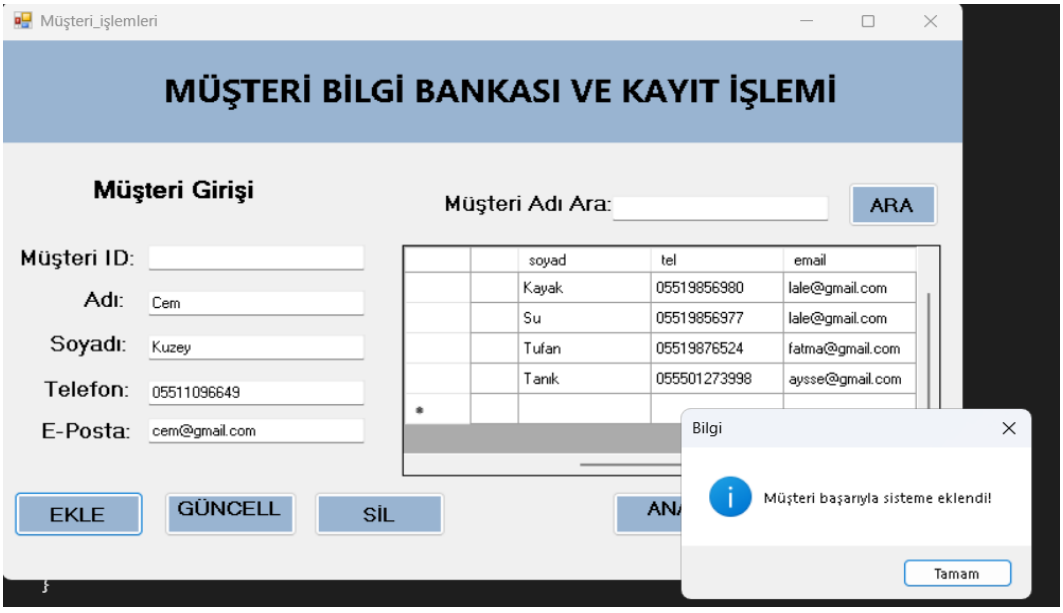


Label: "Barkod No", "Kitap Adı" gibi başlıkları ve açıklamaları göstermek için kullanılan statik metin alanlarıdır.

TextBox: Kullanıcının veri girişi yaptığı (veya verilerin listelendiği) beyaz giriş kutularıdır.

Button (Buton): "EKLE", "GÜNCELL", "SİL", "ARA" gibi işlemleri tetikleyen interaktif bileşenlerdir.

DataGridView: Veritabanından çekilen verilerin satır ve sütunlar halinde liste olarak görüntülendiği tablodur.



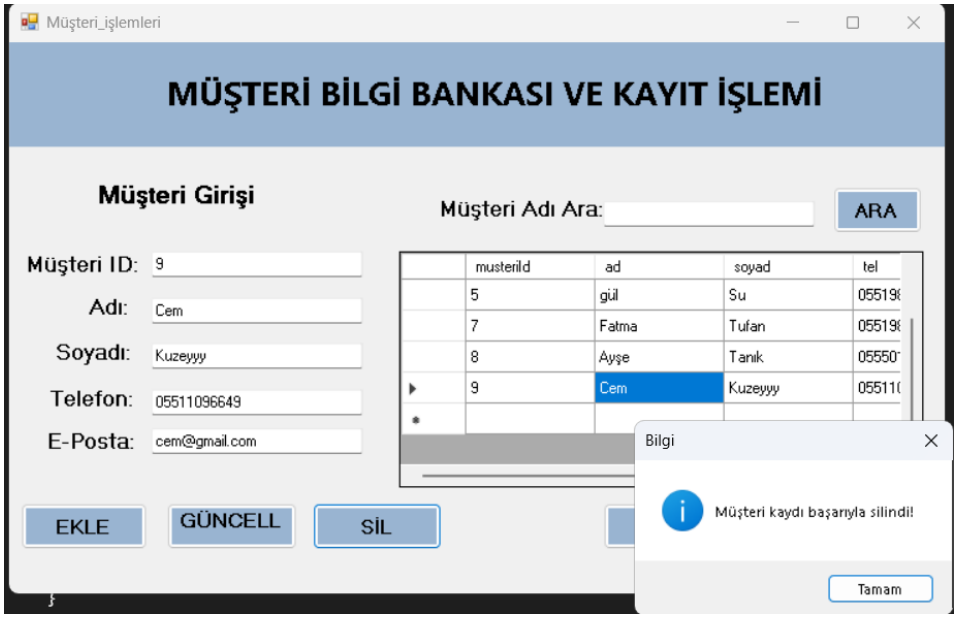
"Form üzerindeki **TextBox** alanlarına girilen müşteri bilgileri, **EKLE** butonuna tıklandığında **INSERT** SQL komutuyla veritabanına gönderilir. Bu işlem, girilen verileri müşteriler tablosuna yeni bir satır olarak kaydeder ve ardından kullanıcıya 'Müşteri başarıyla sisteme eklendi!' bilgilendirme mesajı iletilir."



GÜNCELL butonuna tıklandığında, TextBox alanlarında değiştirilen müşteri verileri UPDATE SQL komutu ile veritabanındaki ilgili kayıtla eşleştirilerek güncellenir. İşlem sonunda, kullanıcının yapılan değişikliği teyit etmesi için 'Müşteri bilgileri başarıyla güncellendi!' bilgilendirme mesajı görüntülenir.

ARA butonuna tıklandığında, 'Müşteri Adı Ara' kutusuna girilen metin ile veri tabanındaki kayıtlar eşleştirilir. Eğer aranan kriterlere uygun bir kayıt bulunamazsa, kullanıcıyı durum hakkında bilgilendirmek için 'Aranan isimde bir müşteri bulunamadı!' uyarı mesajı görüntülenir.





SİL butonuna tıklandığında, seçili olan müşteri kaydı **DELETE** SQL komutu ile veritabanından kalıcı olarak kaldırılır. İşlem sonrası kullanıcıyı bilgilendirmek amacıyla 'Müşteri kaydı başarıyla silindi!' mesajı görüntülenir.

İlgili Kaynak Kod ve Çalışma Mantığı

```
SqlConnection baglanti = new SqlConnection(@"Data Source=BERAT\SQLEXPRESS;Initial Catalog=görsel_proje;Integrated Security=True");
try
{
    baglanti.Open();

    string sorgu = "INSERT INTO musteriler (ad, soyad, tel, email) VALUES (@ad, @soyad, @tel, @mail)";
    SqlCommand komut = new SqlCommand(sorgu, baglanti);

    komut.Parameters.AddWithValue("@ad", mus_ad.Text.Trim());
    komut.Parameters.AddWithValue("@soyad", mus_soyad.Text.Trim());
    komut.Parameters.AddWithValue("@tel", mus_tel.Text.Trim());
    komut.Parameters.AddWithValue("@mail", eposta.Text.Trim());

    komut.ExecuteNonQuery();
    baglanti.Close();

    MessageBox.Show("Müşteri başarıyla sisteme eklendi!", "Bilgi", MessageBoxButtons.OK, MessageBoxIcon.Information);

    Temizle();
    Listele();
}
catch (Exception hata)
{
    MessageBox.Show("Müşteri eklenirken hata oluştu: " + hata.Message, "Hata", MessageBoxButtons.OK, MessageBoxIcon.Error);
    if (baglanti.State == ConnectionState.Open) baglanti.Close();
}
```

Bu kod bloğu, kullanıcı arayüzündeki giriş alanlarına (TextBox) yazılan müşteri bilgilerini, güvenli bir şekilde veritabanındaki musteriler tablosuna ekleyen bir **kayıt işlemi** fonksiyonudur.

İşleyişi:

1. **Bağlantı:** SqlConnection nesnesiyle veritabanına bağlanır.
2. **Sorgu ve Parametreler:** INSERT INTO SQL komutuyla verilerin ekleneceği tablo ve sütunlar belirlenir. @ad, @soyad gibi **parametreler** kullanılarak, dışarıdan gelebilecek zararlı SQL saldırılarına (SQL Injection) karşı güvenlik önlemi alınır.
3. **Veritabanı İşlemi:** ExecuteNonQuery() komutuyla veriler tabloya yazılır ve bağlantı sonlandırılır.
4. **Kullanıcı Geri Bildirimi:** İşlem başarılı olduğunda kullanıcıya bir "Başarı" mesajı gösterilir.
5. **Otomatik Güncelleme:** Temizle() ile giriş kutuları boşaltılır ve Listele() fonksiyonu ile DataGridView üzerindeki veriler yenilenerek eklenen müşteri anında tabloda listelenir.

```
SqlConnection baglanti = new SqlConnection(@"Data Source=BERAT\SQLEXPRESS;Initial Catalog=görsel_proje;Integrated Security=True");
try
{
    baglanti.Open();

    string sorgu = "UPDATE musteriler SET ad=@ad, soyad=@soyad, tel=@tel, email=@mail WHERE musteriId=@id";
    SqlCommand komut = new SqlCommand(sorgu, baglanti);

    komut.Parameters.AddWithValue("@id", id.Text);
    komut.Parameters.AddWithValue("@ad", mus_ad.Text.Trim());
    komut.Parameters.AddWithValue("@soyad", mus_soyad.Text.Trim());
    komut.Parameters.AddWithValue("@tel", mus_tel.Text.Trim());
    komut.Parameters.AddWithValue("@mail", eposta.Text.Trim());

    komut.ExecuteNonQuery();
    baglanti.Close();

    MessageBox.Show("Müşteri bilgileri başarıyla güncellendi!", "Bilgi", MessageBoxButtons.OK, MessageBoxIcon.Information);

    Temizle();
    Listele();
}
catch (Exception hata)
{
    MessageBox.Show("Güncelleme yapılırken hata oluştu: " + hata.Message, "Hata", MessageBoxButtons.OK, MessageBoxIcon.Error);
    if (baglanti.State == ConnectionState.Open) baglanti.Close();
}
```

Bu kod bloğu, kullanıcı arayüzünde (TextBox) yapılan müşteri bilgilerini veritabanındaki musteriler tablosunda **güncellemek** (UPDATE) için kullanılan temel veritabanı işlemidir.

1. **Bağlantı:** SqlConnection ile veritabanına güvenli bir bağlantı açılır.
2. **Sorgu ve Parametreler:** UPDATE SQL komutu ile hangi kayıtların değiştirileceği (WHERE musteriId=@id) ve hangi alanların güncelleneceği (SET ad=@ad, ...) belirlenir. @ işaretli parametreler kullanılarak **SQL Injection** gibi güvenlik açıklarına karşı korunur.
3. **Veritabanı İşlemi:** ExecuteNonQuery() komutuyla güncellenen veriler veritabanına gönderilir ve bağlantı kapatılır.
4. **Kullanıcı Geri Bildirimi:** İşlem başarılı olduğunda "Müşteri bilgileri başarıyla güncellendi!" mesajı gösterilir.
5. **Otomatik Yenileme:** Son adımda Temizle() ile formdaki giriş kutuları sıfırlanır ve Listele() fonksiyonu tekrar çalıştırılarak güncel verilerin tabloya yansıması sağlanır.

```
SqlConnection baglanti = new SqlConnection(@"Data Source=BERAT\SQLEXPRESS;Initial Catalog=görsel_proje;Integrated Security=True");
try
{
    baglanti.Open();


    string sorgu = "DELETE FROM musteriler WHERE musteriId=@id";


    SqlCommand komut = new SqlCommand(sorgu, baglanti);

    komut.Parameters.AddWithValue("@id", id.Text);

    komut.ExecuteNonQuery();
    baglanti.Close();

    MessageBox.Show("Müşteri kaydı başarıyla silindi!", "Bilgi", MessageBoxButtons.OK, MessageBoxIcon.Information);

    Temizle();
    Listele();
}
catch (Exception hata)
{
    MessageBox.Show("Müşteri silinirken hata oluştu: " + hata.Message, "Hata", MessageBoxButtons.OK, MessageBoxIcon.Error);
    if (baglanti.State == ConnectionState.Open) baglanti.Close();
}
```

Bu kod bloğu, kullanıcı tarafından seçilen bir müşteri kaydını **veritabanından kalıcı olarak silmek** için kullanılan DELETE işlemini gerçekleştirmektedir.

İşleyişi:

1. **Bağlantı:** SqlConnection ile SQL Server veritabanına güvenli bir bağlantı açılır.
2. **Sorgu ve Parametreler:** DELETE FROM musteriler WHERE musteriId=@id komutu ile sadece seçilen ID'ye sahip satır hedef alınır. @id parametresi, **SQL Injection** gibi güvenlik açıklarını önlemek için kullanılır.
3. **Veritabanı İşlemi:** ExecuteNonQuery() komutuyla silme işlemi veritabanında uygulanır ve ardından baglanti.Close() ile bağlantı kapatılır.
4. **Kullanıcı Geri Bildirimi:** İşlem başarılı olursa kullanıcıya "Müşteri kaydı başarıyla silindi!" mesajı verilir.
5. **Otomatik Güncelleme:** Temizle() ile giriş kutuları boşaltılır ve Listele() fonksiyonu tekrar çalıştırılarak silinen kaydın artık DataGridView tablosunda görünmemesi sağlanır.

```
private void dataGridView1_CellClick(object sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex >= 0)
    {
        DataGridViewRow satir = dataGridView1.Rows[e.RowIndex];
        id.Text = satir.Cells["musteriId"].Value.ToString();
        mus_ad.Text = satir.Cells["ad"].Value.ToString();
        mus_soyad.Text = satir.Cells["soyad"].Value.ToString();
        mus_tel.Text = satir.Cells["tel"].Value.ToString();
        eposta.Text = satir.Cells["email"].Value.ToString();
    }
}
```

Bu kod bloğu, DataGridView üzerinde herhangi bir satıra **tıklandığında** çalışır ve seçilen satırdaki verileri, düzenleme yapılabilmesi için formdaki ilgili TextBox (metin kutusu) alanlarına otomatik olarak taşır.

İşleyişi:

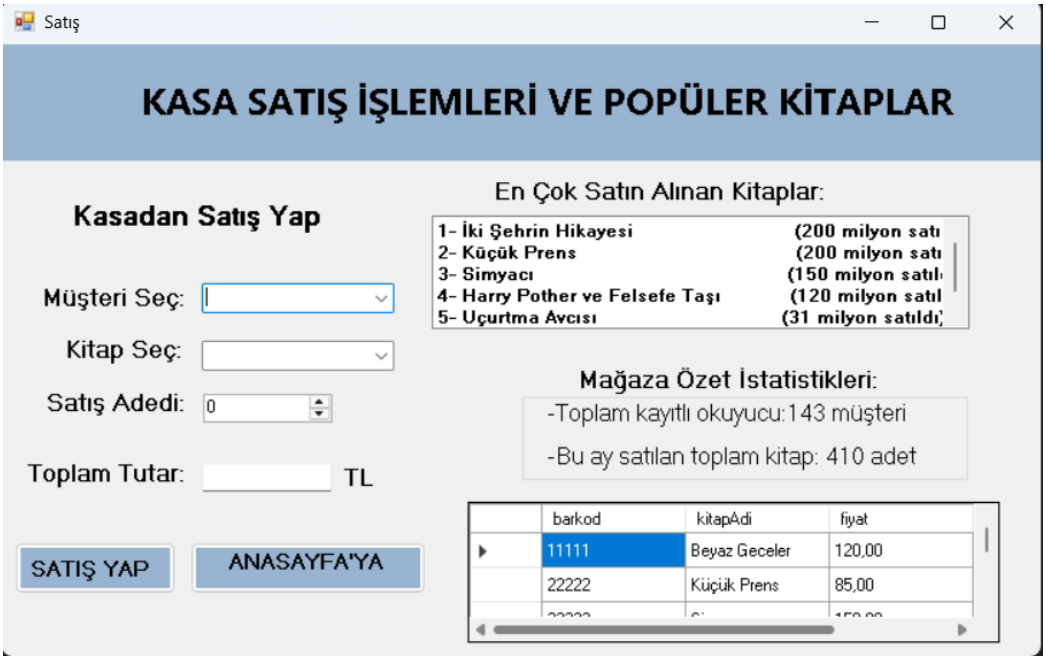
1. **Satır Kontrolü:** if (e.RowIndex >= 0) satırı, kullanıcının boş bir alana değil, geçerli bir veri satırına tıkladığından emin olur.
2. **Veri Okuma:** satir değişkeni ile tıklanan satırın tüm verileri hafızaya alınır.
3. **Aktarım:** Satırın hücrelerindeki bilgiler (musteriId, ad, soyad vb.), karşılık gelen TextBox (id, mus_ad, mus_soyad vb.) alanlarına .Text özelliği kullanılarak atanır.

Veritabanı Yapısı ve SQL İşlemleri

```
CREATE TABLE musteriler (
    musteriID INT IDENTITY PRIMARY KEY,
    ad NVARCHAR(50) NOT NULL,
    soyad NVARCHAR(50) NOT NULL,
    tel NVARCHAR(20),
    email NVARCHAR(100)
);
```

Veritabanında oluşturulan musteriler tablosu, uygulamanın veri deposudur. musteriID sütunu birincil anahtar (Primary Key) olarak tanımlanmış ve verilerin benzersiz olması sağlanmıştır. NOT NULL kısıtlaması ile zorunlu alanlar korunarak veri bütünlüğü garanti altına alınmıştır.

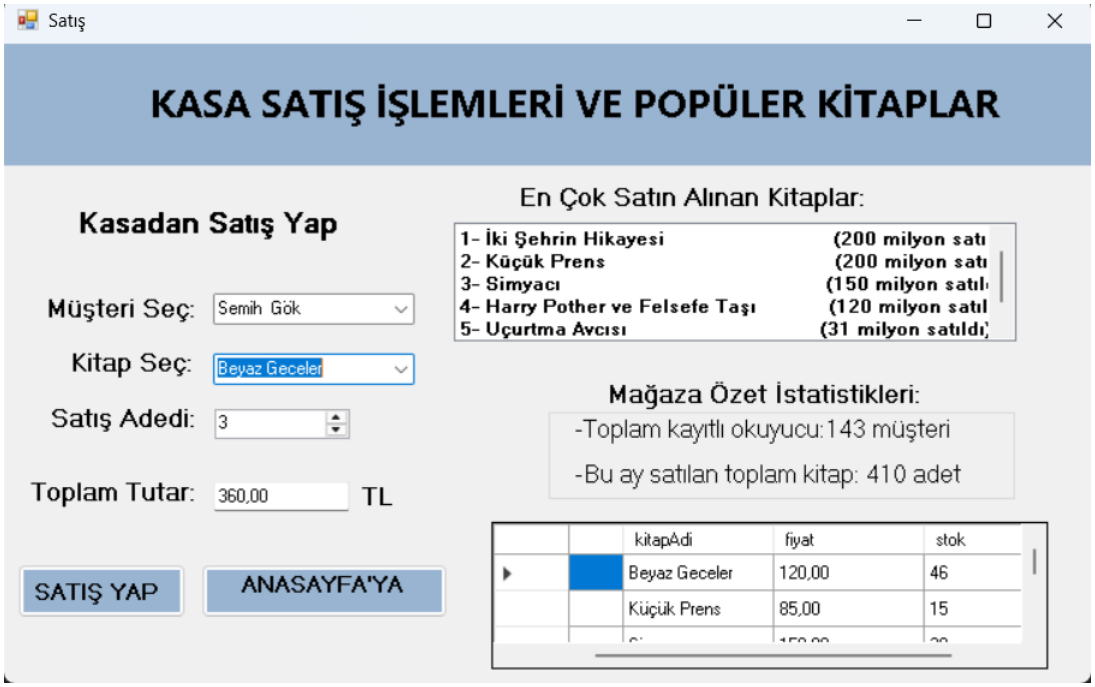
Satış / İstatistik Formu



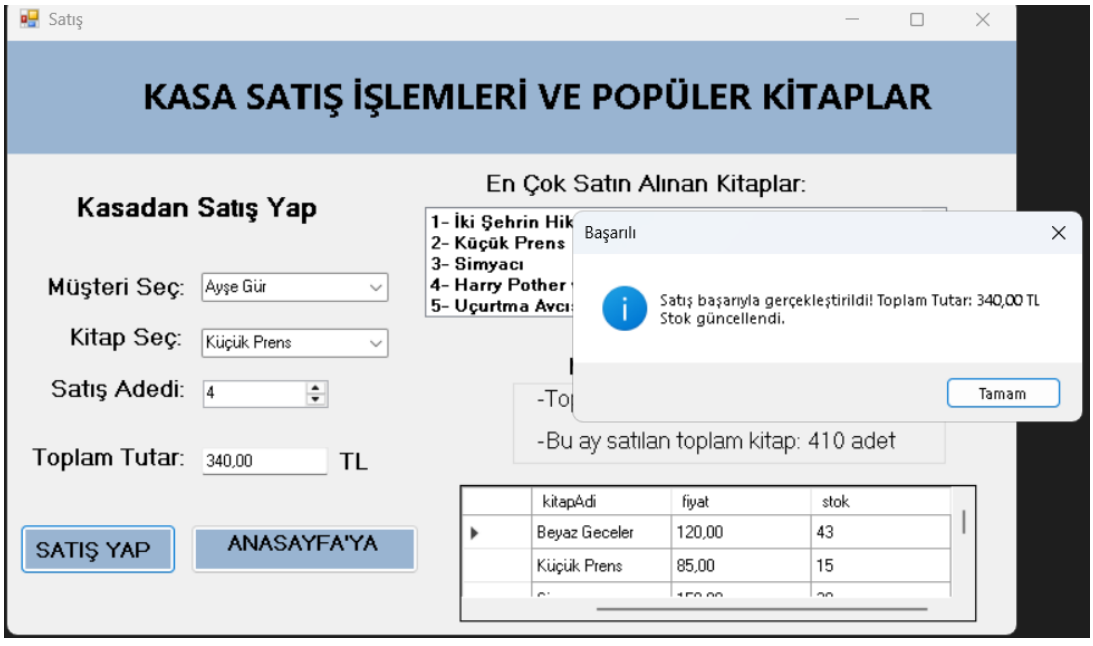
Bu formun amacı, sistemdeki kayıtlı müşterilere kitap satışı gerçekleştirmek ve mağaza verilerini raporlamaktır. Müşteri ve kitap seçimleri yapılarak satış adedi belirlenir, toplam tutar hesaplanır ve işlem veritabanına kaydedilir. Ayrıca, en çok satılan kitaplar ile mağaza genel istatistikleri bu ekran üzerinden takip edilir.

Bu formun arayüzünde şu temel bileşenler kullanılmıştır:

- **ComboBox:** Müşteri ve kitap seçimlerini (listeden seçme) yapmak için kullanılmıştır.
- **NumericUpDown:** Satış adedini hatasız ve sadece sayısal olarak girmek için kullanılmıştır.
- **TextBox:** Toplam tutar bilgisini görüntülemek için (salt okunur veya hesaplanmış veri alanı olarak) kullanılmıştır.
- **Button:** Satış işlemini başlatmak ve ana sayfaya dönmek için kullanılmıştır.
- **Label:** "Müşteri Seç", "Kitap Seç" gibi alanları tanımlayan metin etiketleri olarak kullanılmıştır.
- **DataGridView:** Mevcut kitap stok ve fiyat bilgilerini listelemek için kullanılmıştır.



Bu form, seçilen müşteri ve kitap bilgileri üzerinden hızlı satış işlemleri gerçekleştirmek için tasarlanmıştır. Kullanıcı ComboBox ile müşteri ve kitap seçimi yapıp NumericUpDown ile satış adedini belirler; sistem otomatik olarak toplam tutarı hesaplar. "SATIŞ YAP" butonu ile onaylanan işlem veritabanına kaydedilirken, formun sağ tarafında ise en çok satan kitaplar ve mağaza genel istatistikleri güncel olarak raporlanır.



Bu form, satış işleminin başarıyla tamamlandığını onaylayan geri bildirim ekranıdır. İşlem onaylandığında sistem, veritabanındaki stok miktarını otomatik olarak düşürür, yapılan satışı kaydeder ve kullanıcıya toplam tutar ile stok güncellemesinin gerçekleştiğini belirten bir bilgilendirme mesajı (MessageBox) gösterir.

İlgili Kaynak Kod ve Çalışma Mantığı

```
private void TutarHesapla()
{
    if (cmbkitap.SelectedItem != null && numsatıs.Value > 0)
    {
        SqlConnection baglanti = new SqlConnection(@"Data Source=BERAT\SQLEXPRESS;Initial Catalog=görsel_proje;Integrated Security=True
        try
        {
            if (baglanti.State == ConnectionState.Closed) baglanti.Open();

            SqlCommand komut = new SqlCommand("SELECT fiyat FROM Kitaplar WHERE kitapAdi=@ad", baglanti);
            komut.Parameters.AddWithValue("@ad", cmbkitap.SelectedItem.ToString());
            object sonuc = komut.ExecuteScalar();

            if (sonuc != null)
            {
                decimal kitapFiyati = Convert.ToDecimal(sonuc);
                int adet = Convert.ToInt32(numsatıs.Value);

                toplam.Text = (kitapFiyati * adet).ToString("0.00");
            }
            baglanti.Close();
        }
        catch (Exception)
        {
            if (baglanti.State == ConnectionState.Open) baglanti.Close();
        }
    }
    else
    {
        toplam.Text = "0.00";
    }
}
```

Bu kod, satış ekranındaki **otomatik tutar hesaplama** mantığıdır.

Yaptığı İşlemler:

1. **Veri Çekme:** cmbkitap üzerinden seçilen kitabın fiyatını, SQL sorgusu (SELECT fiyat...) ile veritabanından getirir.
2. **Hesaplama:** ExecuteScalar() ile gelen tekil fiyat değerini decimal tipine dönüştürür ve numsatış (adet) ile çarpar.
3. **Görüntüleme:** Çıkan sonucu, toplam.Text kutusuna formatlı (0.00 TL) bir metin olarak yansıtır.
4. **Güvenlik:** Eğer kitap seçilmemişse veya adet sıfırsa, tutarı otomatik olarak 0.00 yapar.

```
private void Satış_Load(object sender, EventArgs e)
{
    SqlConnection baglanti = new SqlConnection(@"Data Source=BERAT\SQLEXPRESS;Initial Catalog=görsel_proje;Integrated Security=True");
    try
    {

        SqlCommand komutMus = new SqlCommand("SELECT ad + ' ' + soyad AS Musteri FROM Musteriler", baglanti);
        baglanti.Open();
        SqlDataReader okuMus = komutMus.ExecuteReader();
        cmbmus.Items.Clear();
        while (okuMus.Read()) { cmbmus.Items.Add(okuMus["Musteri"].ToString()); }
        baglanti.Close();


        SqlCommand komutKitap = new SqlCommand("SELECT kitapAdi FROM Kitaplar", baglanti);
        baglanti.Open();
        SqlDataReader okuKitap = komutKitap.ExecuteReader();
        cmbkitap.Items.Clear();
        while (okuKitap.Read()) { cmbkitap.Items.Add(okuKitap["kitapAdi"].ToString()); }
        baglanti.Close();


        baglanti.Open();
        SqlDataAdapter da = new SqlDataAdapter("SELECT barkod, kitapAdi, fiyat, stok FROM Kitaplar", baglanti);
        DataTable dt = new DataTable();
        da.Fill(dt);
        dataGridView1.DataSource = dt;
        baglanti.Close();
    }
    catch (Exception) { if (baglanti.State == ConnectionState.Open) baglanti.Close(); }
}
```

Bu kod, **form açıldığı anda** veritabanındaki bilgileri ekranındaki kutulara ve tabloya otomatik olarak yüklemek için kullanılır.

Yaptığı İşlemler:

1. **Müşterileri Listeleme:** Musteriler tablosundaki ad ve soyadları birleştirerek cmbmüs (ComboBox) içine ekler, böylece satış yaparken müşteri seçimi yapabilirsin.
2. **Kitapları Listeleme:** Kitaplar tablosundaki kitap adlarını cmbkitap içine ekler, böylece satılacak kitaplar listeden seçilebilir.
3. **Stokları Gösterme:** Kitaplar tablosundaki tüm verileri (barkod, ad, fiyat, stok) bir DataTable kullanarak dataGridView1 üzerinde görüntüler.

```
private void cmbkitap_SelectedIndexChanged(object sender, EventArgs e)
{
    SqlConnection baglanti = new SqlConnection(@"Data Source=BERAT\SQLEXPRESS;Initial Catalog=görsel_proje;Integrated Security=True");

    if (cmbkitap.SelectedItem != null && numsatıs.Value > 0)
    {
        try
        {
            baglanti.Open();
            SqlCommand komut = new SqlCommand("SELECT fiyat FROM Kitaplar WHERE kitapAdi=@ad", baglanti);
            komut.Parameters.AddWithValue("@ad", cmbkitap.SelectedItem.ToString());
            object sonuc = komut.ExecuteScalar();

            if (sonuc != null)
            {
                decimal kitapFiyati = Convert.ToDecimal(sonuc);
                int adet = Convert.ToInt32(numsatıs.Value);
                toplam.Text = (kitapFiyati * adet).ToString("0.00");
            }
            baglanti.Close();
        }
        catch (Exception)
        {
            if (baglanti.State == ConnectionState.Open) baglanti.Close();
        }

    }
    TutarHesapla();
}
```

Bu kod, **"Kitap seçimi değiştiği anda"** (ComboBox'tan yeni bir kitap seçildiğinde) otomatik olarak tetiklenen bir olaydır.

Yaptığı İşlemler:

1. **Tetikte Bekleme:** Kullanıcı listeden farklı bir kitap seçtiği an bu fonksiyon devreye girer.
2. **Veritabanı Sorgusu:** Seçilen kitabın veritabanındaki güncel fiyatını çeker.
3. **Hesaplama:** TutarHesapla() metodunu çağırarak, yeni seçilen kitabın fiyatını mevcut satış adediyle çarpar ve toplam tutarı günceller.

```
SqlConnection baglanti = new SqlConnection(@"Data Source=BERAT\SQLEXPRESS;Initial Catalog=görsel_proje;Integrated Security=True");
try
{
    baglanti.Open();


    SqlCommand kontrolKomut = new SqlCommand("SELECT stok FROM Kitaplar WHERE kitapAdi=@kitapAd", baglanti);
    kontrolKomut.Parameters.AddWithValue("@kitapAd", cmbkitap.SelectedItem.ToString());
    int mevcutStok = Convert.ToInt32(kontrolKomut.ExecuteScalar());
    int istenenAdet = Convert.ToInt32(numsatıs.Value);

    if (mevcutStok < istenenAdet)
    {
        MessageBox.Show("Yetersiz stok! Depoda sadece " + mevcutStok + " adet bu kitaptan var.", "Uyarı", MessageBoxButtons.OK,
            MessageBoxIcon.Warning);
        baglanti.Close();
        return;
    }


    string stokSorgusu = "UPDATE Kitaplar SET stok = stok - @satilan WHERE kitapAdi=@kitapAd";
    SqlCommand stokKomut = new SqlCommand(stokSorgusu, baglanti);
    stokKomut.Parameters.AddWithValue("@satilan", istenenAdet);
    stokKomut.Parameters.AddWithValue("@kitapAd", cmbkitap.SelectedItem.ToString());
    stokKomut.ExecuteNonQuery();

    MessageBox.Show("Satış başarıyla gerçekleştirildi! Toplam Tutar: " + toplam.Text + " TL\nStok güncellendi.", "Başarılı",
        MessageBoxButtons.OK, MessageBoxIcon.Information);
```

Bu kod, "SATIŞ YAP" butonunun arkasındaki **kritik işlem mantığıdır**.

Yaptığı İşlemler:

1. **Stok Kontrolü:** Önce veritabanına gider, seçilen kitabın stoğunu okur. Eğer depodaki miktar, satılmak istenen adetten azsa, işlemi durdurur ve "Yetersiz stok" uyarısı verir.
2. **Stok Güncelleme:** Stok yeterliyse, UPDATE komutuyla veritabanındaki stok miktarını, satılan adet kadar eksiltir.
3. **Onay Mesajı:** Satış işleminin başarıyla tamamlandığını ve stoğun güncellendiğini belirten bir bilgilendirme penceresi (MessageBox) açar.

```
    SqlDataAdapter da = new SqlDataAdapter("SELECT barkod, kitapAdi, fiyat, stok FROM Kitaplar", baglanti);
    DataTable dt = new DataTable();
    da.Fill(dt);
    dataGridView1.DataSource = dt;

    baglanti.Close();


    cmbkitap.SelectedIndex = -1;
    cmbmus.SelectedIndex = -1;
    numsatıs.Value = 0;
    toplam.Text = "0.00";
}
catch (Exception hata)
{
    MessageBox.Show("Satış yapılırken hata oluştu: " + hata.Message, "Hata", MessageBoxButtons.OK, MessageBoxIcon.Error);
    if (baglanti.State == ConnectionState.Open) baglanti.Close();
}
```

Bu kod, satış işlemi bittikten sonra ekranı temizlemek ve güncel verileri listelemek için kullanılan **temizlik ve güncelleme** bloğudur.

Yaptığı İşlemler:

1. **Tabloyu Yenileme:** SqlDataAdapter ile veritabanından güncel stok verilerini tekrar çeker ve dataGridView1 üzerinde gösterir; böylece kullanıcı, azalan stoğu anında görür.
2. **Formu Sıfırlama:** Satış sonrası ComboBox seçimlerini kaldırır (SelectedIndex = -1), adet kutusunu sıfırlar ve toplam tutarı tekrar "0.00" yapar.
3. **Hata Yönetimi:** İşlem sırasında herhangi bir sorun olursa, catch bloğu devreye girer; hatayı kullanıcıya bir mesaj kutusu ile bildirir ve açık kalan veritabanı bağlantısını güvenli bir şekilde kapatır.

Veritabanı Yapısı ve SQL İşlemleri

```
CREATE TABLE satislar (
    satisID INT IDENTITY PRIMARY KEY,
    musteriID INT NOT NULL,
    kitapID INT NOT NULL,
    adet INT NOT NULL,
    toplamTutar DECIMAL(18,2) NOT NULL,
    satisTarihi DATETIME DEFAULT GETDATE()
);

SELECT * FROM personeller;
SELECT * FROM  kitaplar;
SELECT * FROM  musteriler;
SELECT * FROM satislar;
```

Bu kod, projenin **veritabanı altyapısını** tanımlar ve mevcut tabloları kontrol etmeni sağlar.

Kodun İşlevi:

1. **CREATE TABLE satislar**: Satış işlemlerini kalıcı olarak kaydetmek için yeni bir tablo oluşturur.
    - **satisID**: Her satışa benzersiz bir numara verir.
    - **musteriID, kitapID, adet**: Hangi müşterinin, hangi kitabı, kaç adet aldığını tutar.
    - **toplamTutar**: İşlemin maddi değerini saklar (DECIMAL(18,2) hassas para hesaplaması içindir).
    - **satisTarihi**: İşlemin yapıldığı anı GETDATE() fonksiyonu ile otomatik kaydeder.
2. **SELECT * FROM ...**: Veritabanındaki personeller, kitaplar, musteriler ve satislar tablolarının içindeki tüm kayıtları ekrana listeler; verilerin durumunu kontrol etmeni sağlar.

```
use görsel_proje;

DROP TABLE IF EXISTS satislar;
DROP TABLE IF EXISTS musteriler;
DROP TABLE IF EXISTS kitaplar;
DROP TABLE IF EXISTS personeller;
```

Bu kod, veritabanı temizleme ve hazırlık aşamasıdır.

İşlevi:

1. **USE görsel_proje**: İşlemlerin görsel_proje adlı veritabanı üzerinde yapılacağını belirtir.
2. **DROP TABLE IF EXISTS ...**: Eğer varsa; satışlar, müşteriler, kitaplar ve personeller tablolarını veritabanından **tamamen siler.**

Proje Özeti

Bu çalışma kapsamında, bir kitabevinin operasyonel süreçlerini dijitalleştirmek amacıyla "Kitap Satış Takip Sistemi" geliştirilmiştir. Proje, satış işlemleri, stok yönetimi ve veritabanı entegrasyonu süreçlerinin manuel hatalardan arındırılarak hatasız bir şekilde yürütülmesini sağlamıştır.